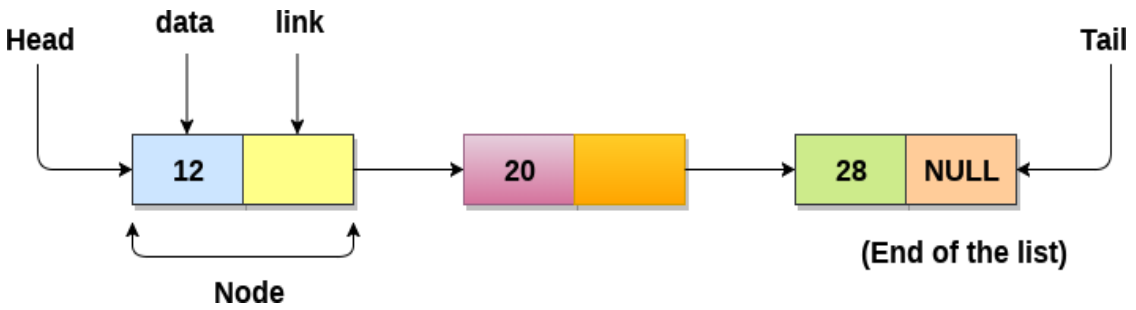
Linked List

- Linked List can be defined as collection of objects called **nodes** that are randomly stored in the memory.
- A node contains two fields i.e. data stored at that particular address and the pointer which contains the address of the next node in the memory.
- The last node of the list contains pointer to the null.



Uses of Linked List

- The list is not required to be contiguously present in the memory. The node can reside anywhere in the memory and linked together to make a list. This achieves optimized utilization of space.
- list size is limited to the memory size and doesn't need to be declared in advance.
- Empty node cannot be present in the linked list.
- We can store values of primitive types or objects in the singly linked list.

Why use linked list over array?

Till now, we were using array data structure to organize the group of elements that are to be stored individually in the memory. However, Array has several advantages and disadvantages which must be known in order to decide the data structure which will be used throughout the program.

Array contains following limitations:

1. The size of array must be known in advance before using it in the program.
2. Increasing size of the array is a time taking process. It is almost impossible to expand the size of the array at run time.
3. All the elements in the array need to be contiguously stored in the memory. Inserting any element in the array needs shifting of all its predecessors.

Linked list is the data structure which can overcome all the limitations of an array. Using linked list is useful because,

1. It allocates the memory dynamically. All the nodes of linked list are non-contiguously stored in the memory and linked together with the help of pointers.
2. Sizing is no longer a problem since we do not need to define its size at the time of declaration. List grows as per the program's demand and limited to the available memory space.

Singly linked list or One-way chain

Singly linked list can be defined as the collection of ordered set of elements. The number of elements may vary according to need of the program. A node in the singly linked list consist of two parts: data part and link part. Data part of the node stores actual information that is to be represented by the node while the link part of the node stores the address of its immediate successor.

One-way chain or singly linked list can be traversed only in one direction. In other words, we can say that each node contains only next pointer, therefore we cannot traverse the list in the reverse direction.

Consider an example where the marks obtained by the student in three subjects are stored in a linked list as shown in the figure.

In the above figure, the arrow represents the links. The data part of every node contains the marks obtained by the student in the different subject. The last node in the list is identified by the null pointer which is present in the address part of the last node. We can have as many elements we require, in the data part of the list.

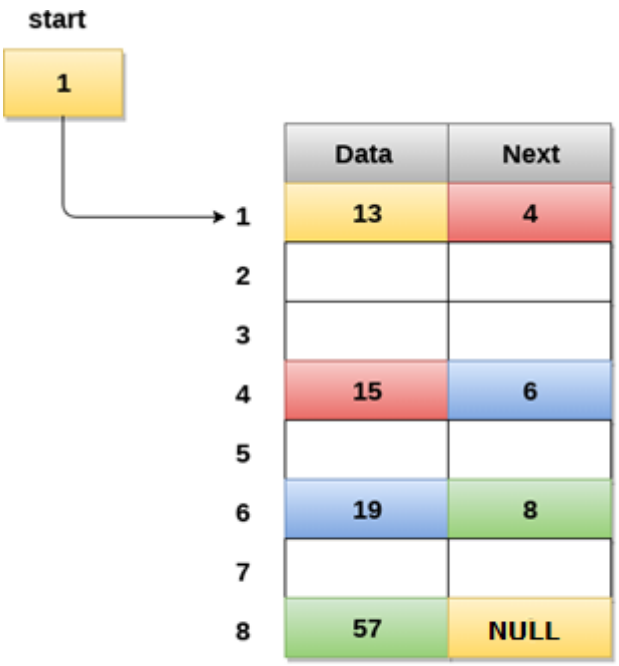
Complexity

Data Structure	Time Complexity								Space Compleity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Singly Linked List	θ(n)	θ(n)	θ(1)	θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)

Memory Representation of Singly Linked List:

In the following image, memory representation of a singly linked list containing marks of a student in 4 subjects. However, the image shows a glimpse of how the singly linked list is being stored in the memory. The start or head of the list is pointing to the element with the index 1 and containing 13 marks in the data part and 4 in the next part. Which means that it is linked with the node that is being stored at 4th index of the list.

However, due to the fact that we are considering circular linked list in the memory therefore the address partition of last node of the list contains the **NULL** value.



Memory representation of s Singly Linked List

Operations on Singly Linked List

There are various operations which can be performed on singly linked list. A list of all such operations is given below.

Node Creation

```
struct node
{
    int data;
    struct node *next;
};
struct node *head, *ptr;
ptr = (struct node *) malloc(sizeof(struct node *));
```

Insertion

The insertion into a singly linked list can be performed at different positions. Based on the position of the new node being inserted, the insertion is categorized into the following categories.

SN	Operation	Description
1	Insertion at beginning	It involves inserting any element at the front of the list. We just need to a few link adjustments to make the new node as the head of the list.
2	Insertion at end of the list	It involves insertion at the last of the linked list. The new node can be inserted as the only node in the list or it can be inserted

		as the last one. Different logics are implemented in each scenario.
3	Insertion after specified node	It involves insertion after the specified node of the linked list. We need to skip the desired number of nodes in order to reach the node after which the new node will be inserted.

Deletion and Traversing

The Deletion of a node from a singly linked list can be performed at different positions. Based on the position of the node being deleted, the operation is categorized into the following categories.

SN	Operation	Description
1	Deletion at beginning	It involves deletion of a node from the beginning of the list. This is the simplest operation among all. It just need a few adjustments in the node pointers.
2	Deletion at the end of the list	It involves deleting the last node of the list. The list can either be empty or full. Different logic is implemented for the different scenarios.
3	Deletion after specified node	It involves deleting the node after the specified node in the list. we need to skip the desired number of nodes to reach the node after which the node will be deleted. This requires traversing through the list.
4	Traversing	In traversing, we simply visit each node of the list at least once in order to perform some specific operation on it, for example, printing data part of each node present in the list.
5	Searching	In searching, we match each element of the list with the given element. If the element is found on any of the location then location of that element is returned otherwise null is returned.

Singly Linked List: Menu Driven Program

```
#include<stdio.h>
#include<stdlib.h>
strtct Node
{
        int data;
        struct Node *next;
};
struct Node *start = NULL, *end = NULL, *temp;
void insertEnd ( int ele)
{
        temp = (struct Node *) malloc ( sizeof (struct Node )
        ); temp->data = ele;
        temp->next = NULL;
```

```
if ( start == NULL )            // When list is not having any nodes.
```

```
        {
                start = end = temp;
        }
        else            // When list is having atleast one node.
        {
                end->next = temp;
                end = temp;
        }
}
void deleteEnd ( )
{
        if ( start == NULL )            // When list is empty.
        {
                printf ( " The list is empty ");
        }
        else if (start == end )         // When list is Having only one node.
        {
                start = end = NULL:
        }
        else            // When list is having more than one node.
        {
                temp = start;
                while ( temp->next != end )
                {
                        temp = temp->next;
                }
                end = temp;
                end->next = NULL;
        }
}
void insertBeginning ( int ele )
{
        temp = (struct Node *) malloc ( sizeof (struct Node ) );
        temp->data = ele;
        temp->next = NULL;
        if ( start == NULL )            // When list is not having any nodes.
        {
                start = end = temp;
        }
        else            // When list is having atleast one node.
        {
                temp->next = start;
                start = temp;
        }
}
void deleteBeginning ( )
{
        if ( start == NULL )            // When list is empty.
        {
                printf ( " The list is empty ");
        }
        else  if (start == end )        // When list is Having only one node.
        {
                start = end = NULL:
```

```
        }
        else            // When list is having more than one node.
        {
                temp = start;           // Assiging first node to temp.
                start = start->next;
                free ( temp );          // Deleting first node from memory.
        }

}
void insertAtPosition ( int ele, int pos )
{
        struct Node *curr;
        int i;
        if ( pos == 1)
        {
                                        insertBeginning ( ele );
        }
        else
        {
                                        temp = (struct Node *)
                                        malloc ( sizeof (struct
                                        Node ) ); temp->data =
                                        ele;
                                        temp->next = NULL;

                                        curr = start;
                                        for ( i=1; i<pos-1; i++ )       // Moving to
                                        previous of target position
                                        {
                        curr = curr->next;
                }
                if ( curr == NULL )
                {
                                        printf ( " \n Insertion not possible, Position
                }                       out of range " );
                else
                {

                                        te
                                        m
                                        p-
                                }       >n
                        }               ext
                }                       =
                                        cu
                                        rr-
                                        >n
                                        ext
                                        ;
                                        cu
                                        rr-
                                        >n
                                        ext
                                        =
```

```
te                                                  if ( curr == end )
m                                                           end = temp;
p;
        void deleteAtPosition ( int pos )
        {
                int i;
                struct Node *curr;
                if ( start == NULL )
                {
                        printf ( " List is empty ");
                }
                else if ( pos == 1)
                {
                        deleteBeginning ( );
                }
                else
```

```
        {
                curr = start;
                for ( i=1; i<pos-1; i++ )       // Moving to previous of target position
                {
                        curr = curr ->next;
                }
                if ( curr == NULL )
                {
                        printf ( " \n Deletion not possible, Position out of range " );
                }
                else
                {       if ( curr ->next == end )
                        {
                                                end = curr;
                                                end->next = NULL:
                        }
                        else
                                                curr ->next = curr ->next->next;
                }
        }
}

void displayList ( )
{
        if ( start == NULL )
        {
                                                printf ( " List is empty ");
        }
        else
        {
                                                temp = start;
                                                while ( temp != NULL)                   //
                                                Diplaying all nodes except last one.
                                                {
                        printf ( " %d -> ", temp->data );
                        temp = temp -> next;
                }
        }
}
void searchKey ( int key )
{
        if ( start == NULL )
        {
                                                printf ( " List is empty " );
        }
        else
        {
                                                i
                                                n
                                                t
                                                p
                                                o
                                                s
                                                =
```

```
0
;
t
e
m
p
=
s
t
a
r
t
;
while ( temp != NULL )
{
pos++;
if ( temp->data = key )
{
        printf ( " \n Key node found at the position: %d. ", pos);
        break;
```

```
                        }
                        temp = temp -> next;
                }
                if ( temp == NULL )
                {
                        printf ( " \n Key element not found in the list. ");
                }
        }
}
int main ( )
{
        int choice, ele, pos, key ;
        while ( 1 )
        {
                printf ( " \n Single Linked List Operations \n ");

                printf( " 1. Insert End  \n ");
                printf( " 2. Delete End \n ");
                printf( " 3. Insert Beginning \n ");
                printf( " 4. Delete Beginning \n ");
                printf( " 5. Insert At Position \n
                ");
                printf( " 6. Delete At Position \n ");
                printf( " 7. Display List \n ");
                printf( " 8. Search Key \n ");
                printf( " 9. Exit\n ");

                printf ( " Enter your choice: " );
                scanf ( " %d " , &choice );

                switch ( choice )
                {
                        case 1 :
                                printf ( " Enter the element: "
                                ); scanf ( " %d " , &ele);
                                insertEnd ( ele );
                                break;
                        case 2 :
                                deleteEnd ( );
                                break;
                        case 3 :
                                printf ( " Enter the element: " );
                                scanf ( " %d " , &ele);
                                insertBeginning ( ele );
                                break;
                        case 4 :
                                deleteBeginning ( );
                                break;
                        case 5 :
                                printf ( " Enter the element: " );
                                scanf ( " %d " , &ele);
                                printf ( " Enter the position : " );
                                scanf ( " %d ", &pos);
                                insertAtPosition ( ele, pos );
                                break;
                        case 6 :
                                printf (" Enter the position: " );
                                scanf ( " %d ", &pos);
```

```
                        deleteAtPosition ( pos );
                        break;
                case 7 :
                        displayList ( );
                        break;
                case 8 :
                        printf ( “ Enter the key element to find: “ );
                        scanf ( “ %d “, &key);
                        searchKey ( key );
                        break;
                case 9 :
                default :
                        return 0;
            }
        }
}
```

Merging two Singly Linked Lists in to one List

```
#include<stdio.h>
#include<stdlib.h>
struct Node
{
        int data;
        struct Node *next;
};
struct Node *list1, *list2, *temp, *start, *end;
struct Node* createList()
{
        char ch;
        start = end = NULL;
        while (1)
        {
                temp = (struct Node*) malloc (sizeof(struct Node));
                printf("\nEnter node value: ");
                scanf("%d",
                &temp->data);
                temp->next = NULL;
                if(start == NULL)
                {
                                        start = end = temp;
                }
                else
                {
                                        e
                                        n
                }                       d
                                        -
                                        >
                                        n
                                        e
                                        x
```

```
t                                   n
=                                   d
t                                   =
e                                   t
m                                   e
p                                   m
;                                   p
e                                   ;
                printf("Do you want to continue (y/n) : ");
                fflush(stdin);
                scanf("%c", &ch);
                if(ch == 'y' || ch == 'Y')
                        continue;
                else
```

```
                        break;
        }
        return start;
}
void displayList(struct Node *list)
{
        if(list == NULL)
        {
                                printf(" List is empty ");
        }
        else
        {
                                t
                                e
                                m
                                p
                                =
                                l
                                i
                                s
                                t
                                ;
                                w
                                h
                                i
                                l
                                e
                                (
                                t
                                e
                                m
                                p
                                !
                                =
                                N
                                U
                                L
                                L
                                )
                                {
                        printf("%d->", temp->data);
                        temp = temp->next;
                }
                printf("NULL");
        }
}
void concatenate()
{
        if(list1 != NULL && list2 != NULL)
        {
                temp = list1;
                while(temp->next != NULL)
                {
                        temp = temp->next;
```

```
                }
                                        temp->next=list2;
        }
        else
        {
                                        display List(list1);
        }
                }
                                        printf("\nList is empty");
int main()
{
        clrscr();
        printf("\n Create List-1: ");
        list1 = createList();
        printf("\n Create List-2: ");
        list2 = createList();
        printf("\n List-1 : ");
        displayList(list1);
        printf("\n List-2 : ");
        displayList(list2);
        printf("\n List1 + List2: ");
        concatenate();
        return 0;
```

}

Reversing a Singly Linked List

```
#include<stdio.h>
#include<stdlib.h>
int stack[50], top = -1;
void push(int ele)
{
        stack[++top] = ele;
}
int pop()
{
        return stack[top--];
}
struct Node
{
        int data;
        struct Node *next;
};
struct Node* createList(struct Node *start)
{
        struct Node *temp, *curr;
        char ch;
        do
        {
                int val;
                printf("\n Enter node value: ");
                scanf("%d", &val);
                temp = (struct Node*)malloc(sizeof(struct Node));
                temp->data = val;
                temp->next = NULL;
                if(start == NULL)
                {
                                start = curr = temp;
                }
                else
                {
                                curr = start;
                                whil
                                        e
                                        (
                }                       c
                                        u
                                        r
                                        r
                                        -
                                        >
                                        n
                                        e
```

```
x                                                    =
t                                                    c
!                                                    u
=                                                    r
N                                                    r
U                                                    -
L                                                    >
L                                                    n
)                                                    e
c                                                    x
u                                                    t
r                                                    ;
r                                                curr->next = temp;
                printf("\n Do you want to insert new node? (y/n): ");
                scanf(" %c", &ch);
        }while(ch == 'y' || ch == 'Y');
        return start;
}
void display(struct Node *start)
{
```

```
if(start == NULL)
{
        printf("\n List is empty \n");
}
else
{
        s
        t
        r
        u
        c
        t
        N
        o
        d
        e
        *
        t
        e
        m
        p
        ;
        t
        e
        m
        p
        =
        s
        t
        a
        r
        t
        ;
        w
        h
        i
        l
        e
        (
        t
        e
        m
        p
        !
        =
        N
        U
        L
```

```
L                                                    )
                                                     {
                        printf("%d->", temp->data);
                        temp = temp->next;
                    }
                    printf("NULL");
            }
    }
    struct Node* reverseList(struct Node *start)
    {
            if(start == NULL)
            {
                                                     printf("\n List is empty \n");
            }
            else
            {
                                                     s
                                                     t
                                                     r
                                                     u
                                                     c
                                                     t
                                                     N
                                                     o
                                                     d
                                                     e
                                                     *
                                                     t
                                                     e
                                                     m
                                                     p
                                                     ;
                                                     t
                                                     e
                                                     m
                                                     p
                                                     =
                                                     s
                                                     t
                                                     a
                                                     r
                                                     t
                                                     ;
                                                     w
                                                     h
                                                     i
                                                     l
                                                     e
```

```
(
t
e
m
p
!
=
N
U
L
L
)
{
push(temp->data);
temp = temp->next;
}
temp = start;
while(temp != NULL && top != -1)
{
temp->data = pop();
temp = temp->next;
}
}
return start;
}
int main()
{
struct Node *start = NULL;
printf("\n Create the list: \n");
start = createList(start);
printf("\n Creted list: ");
display(start);
```

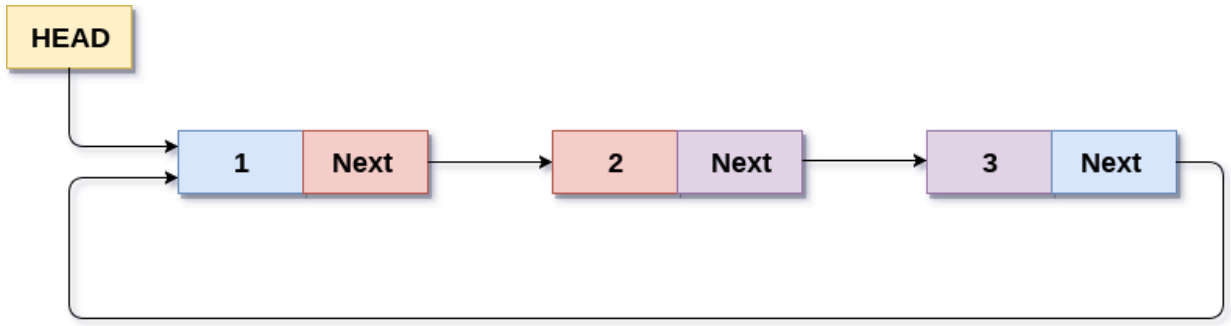
```
        start = reverseList(start);
        printf("\n List after reverse: ");
        display(start);
        return 0;
}
```

Circular Singly Linked List

In a circular Singly linked list, the last node of the list contains a pointer to the first node of the list. We can have circular singly linked list as well as circular doubly linked list.

We traverse a circular singly linked list until we reach the same node where we started. The circular singly liked list has no beginning and no ending. There is no null value present in the next part of any of the nodes.

The following image shows a circular singly linked list.



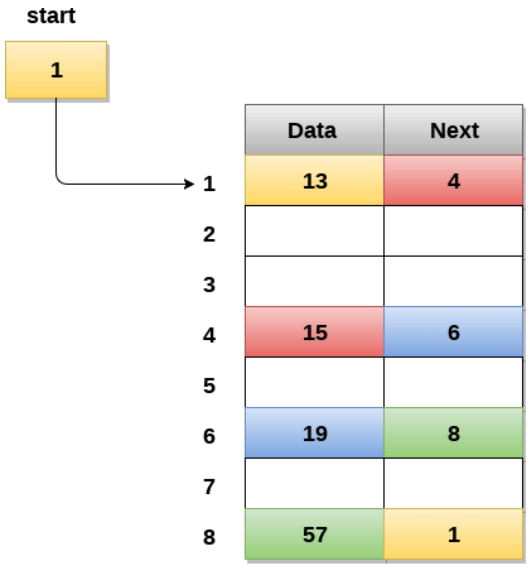
Circular Singly Linked List

Circular linked list are mostly used in task maintenance in operating systems. There are many examples where circular linked list are being used in computer science including browser surfing where a record of pages visited in the past by the user, is maintained in the form of circular linked lists and can be accessed again on clicking the previous button.

Memory Representation of circular linked list:

In the following image, memory representation of a circular linked list containing marks of a student in 4 subjects. However, the image shows a glimpse of how the circular list is being stored in the memory. The start or head of the list is pointing to the element with the index 1 and containing 13 marks in the data part and 4 in the next part. Which means that it is linked with the node that is being stored at 4th index of the list.

However, due to the fact that we are considering circular linked list in the memory therefore the last node of the list contains the address of the first node of the list.



Memory Representation of a circular linked list

We can also have more than one number of linked list in the memory with the different start pointers pointing to the different start nodes in the list. The last node is identified by its next part which contains the address of the start node of the list. We must be able to identify the last node of any linked list so that we can find out the number of iterations which need to be performed while traversing the list.

Operations on Circular Singly linked list:

Insertion

SN	Operation	Description
1	Insertion at beginning	Adding a node into circular singly linked list at the beginning.
2	Insertion at the end	Adding a node into circular singly linked list at the end.

Deletion & Traversing

SN	Operation	Description
1	Deletion at beginning	Removing the node from circular singly linked list at the beginning.

2	Deletion at the end	Removing the node from circular singly linked list at the end.
3	Searching	Compare each element of the node with the given item and return the location at which the item is present in the list otherwise return null.
4	Traversing	Visiting each element of the list at least once in order to perform some specific operation.

Menu-driven program: CLL

```
#include<stdio.h>
#include<stdlib.h>
strtct Node
{
        int data;
        struct Node *next;
};
struct Node *start = NULL, *end = NULL, *temp;
void insertEnd ( int ele)
{
        temp = (struct Node *) malloc ( sizeof (struct Node )
        ); temp->data = ele;
        temp->next = NULL;
        if ( start == NULL )            // When list is not having any nodes.
        {
                start = end = temp;
        }
        else            // When list is having atleast one node.
        {
                end->next = temp;
                end = temp;
        }
        end->next = start;              // Linking last node to first
}
void deleteEnd ( )
{
        if ( start == NULL )            // When list is empty.
        {
                printf ( " The list is empty ");
        }
        else if (start == end )         // When list is Having only one node.
        {
                start = end = NULL:
        }
        else            // When list is having more than one node.
        {
                temp = start;
                while ( temp->next != end )
```

}

```
                        temp = temp->next;
                }
                end = temp;
                end->next = NULL;
        }
        end->next = start;
}
void insertBeginning ( int ele )
{
        temp = (struct Node *) malloc ( sizeof (struct Node ) );
        temp->data = ele;
        temp->next = NULL;
        if ( start == NULL )            // When list is not having any nodes.
        {
                start = end = temp;
        }
        else            // When list is having atleast one node.
        {
                temp->next = start;
                start = temp;
        }
        end->next = start;
}
void deleteBeginning ( )
{
        if ( start == NULL )            // When list is empty.
        {
                printf ( " The list is empty ");
        }
        else if (start == end )         // When list is Having only one node.
        {
                start = end = NULL:
        }
        else            // When list is having more than one node.
        {
                temp = start;           // Assiging first node to temp.
                start = start->next;
                free ( temp );          // Deleting first node from memory.
        }
        end->next = start;
}
void insertAtPosition ( int ele, int pos )
{
        struct Node *curr;
        int i;
        if ( pos == 1)
        {
                                        insertBeginning ( ele );
        }
        else
        {
                                        temp = (struct Node *)
                                        malloc ( sizeof (struct
                                        Node ) ); temp->data =
```

```
ele;
temp->next = NULL;
```

```
                curr = start;
                for ( i=1; i<pos-1; i++ )       // Moving to previous of target position
                {
                        curr = curr->next;
                }
                if ( curr == NULL )
                {
                                                printf ( " \n Insertion not possible, Position
                }                               out of range " );
                else
                {

                                                te
                                                m
                                                p-
                }                               >n
                      }                         ext
                                                =
                                                cu
                                                rr-
                                                >n
                                                ext
                                                ;
                                                cu
                                                rr-
                                                >n
                                                ext
                                                =
                                                te
                                                m
                                                p;
                                                if ( curr == end )
                                                        end = temp;
        end->next = start;
}
void deleteAtPosition ( int pos )
{
        int i;
        if ( start == NULL )
        {
                printf ( " List is empty ");
        }
        else if ( pos == 1)
        {
                                                deleteBeginning ( );
        }
        else
        {
                                                temp = start;
                                                for ( i=1; i<pos-1; i++ )       // Moving to
                                                previous of target position
                                                {
                        temp = temp->next;
                }
```

```
                if ( temp == start )
                {
                        printf ( " \n Deletion not possible, Position out of range " );
                }
                else
                {       if ( temp->next == end )
                        {
                                                end = temp;
                                                end->next = NULL:
                        }
                        else
                                                temp->next = temp->next->next;
                         }
                 }

        end->next = start;
}
```

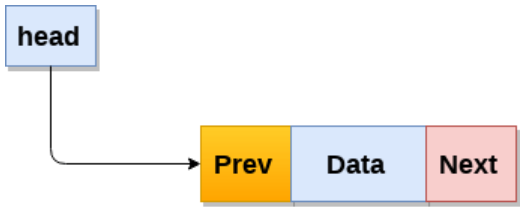
```
void displayList ( )
{
        if ( start == NULL )
        {
                                        printf ( " List is empty ");
        }
        else
        {
                                        temp = start;
                                        while ( temp != end)          // Diplaying all
                                        nodes except last one.
                                        {
                printf ( " %d -> ", temp->data );
                temp = temp -> next;
            }
            printf ( " %d ", temp->data );        // To display the last node.
        }
}
int main ( )
{
        int choice, ele, pos, key ;
        while ( 1 )
        {
            printf ( " \n Circular Linked List Operations \n ");

            printf( " 1. Insert End  \n ");
            printf( " 2. Delete End \n ");
            printf( " 3. Insert Beginning \n ");
            printf( " 4. Delete Beginning \n ");
            printf( " 5. Insert At Position \n
            ");
            printf( " 6. Delete At Position \n ");
            printf( " 7. Display List \n ");
            printf( " 8. Exit\n ");

            printf ( " Enter your choice: " );
            scanf ( " %d " , &choice );

            switch ( choice )
            {
                case 1 :
                    printf ( " Enter the element: "
                    ); scanf ( " %d " , &ele);
                    insertEnd ( ele );
                    break;
                case 2 :
                    deleteEnd ( );
                    break;
                case 3 :
                    printf ( " Enter the element: " );
                    scanf ( " %d " , &ele);
                    insertBeginning ( ele );
                    break;
                case 4 :
                    deleteBeginning ( );
                    break;
                case 5 :
```

```
printf ( " Enter the element: " );
scanf ( " %d " , &ele);
```

```
                        printf ( " Enter the position : " );
                        scanf ( " %d ", &pos);
                        insertAtPosition ( ele, pos );
                        break;
                case 6 :
                        printf (" Enter the position: " );
                        scanf ( " %d ", &pos);
                        deleteAtPosition ( pos );
                        break;
                case 7 :
                        displayList ( );
                        break;
                case 8 :
                default :
                        return 0;
            }
        }
    }
```

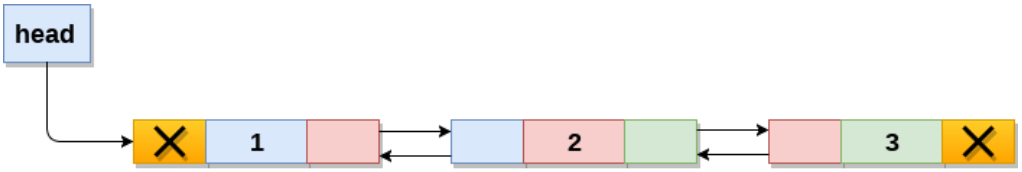
Doubly linked list

Doubly linked list is a complex type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence. Therefore, in a doubly linked list, a node consists of three parts: node data, pointer to the next node in sequence (next pointer) , pointer to the previous node (previous pointer). A sample node in a doubly linked list is shown in the figure.



Node

A doubly linked list containing three nodes having numbers from 1 to 3 in their data part, is shown in the following image.



Doubly Linked List

In C, structure of a node in doubly linked list can be given as :

```
struct node
```

```
{
    struct node *prev;
    int data;
    struct node *next;
}
```

The **prev** part of the first node and the **next** part of the last node will always contain null indicating end in each direction.

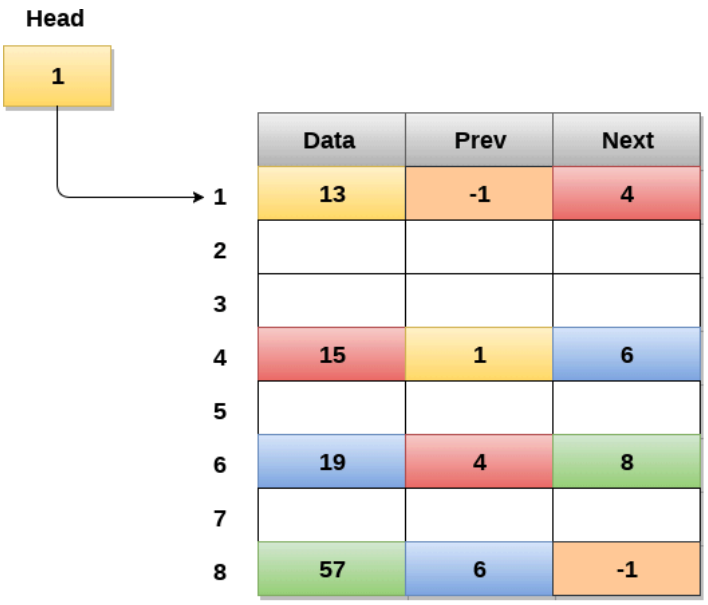
In a singly linked list, we could traverse only in one direction, because each node contains address of the next node and it doesn't have any record of its previous nodes. However, doubly linked list overcome this limitation of singly linked list. Due to the fact that, each node of the list contains the address of its previous node, we can find all the details about the previous node as well by using the previous address stored inside the previous part of each node.

Memory Representation of a doubly linked list

Memory Representation of a doubly linked list is shown in the following image. Generally, doubly linked list consumes more space for every node and therefore, causes more expansive basic operations such as insertion and deletion. However, we can easily manipulate the elements of the list since the list maintains pointers in both the directions (forward and backward).

In the following image, the first element of the list that is i.e. 13 stored at address 1. The head pointer points to the starting address 1. Since this is the first element being added to the list therefore the **prev** of the list **contains** null. The next node of the list resides at address 4 therefore the first node contains 4 in its next pointer.

We can traverse the list in this way until we find any node containing null or -1 in its next part.



Memory Representation of a Doubly linked list

Operations on doubly linked list

Node Creation

```
struct node
{
    struct node *prev;
    int data;
    struct node *next;
};
struct node *head;
```

All the remaining operations regarding doubly linked list are described in the following table.

SN	Operation	Description
1	Insertion at beginning	Adding the node into the linked list at beginning.
2	Insertion at end	Adding the node into the linked list to the end.
3	Insertion after specified node	Adding the node into the linked list after the specified node.
4	Deletion at beginning	Removing the node from beginning of the list
5	Deletion at the end	Removing the node from end of the list.
6	Deletion of the node having given data	Removing the node which is present just after the node containing the given data.
7	Searching	Comparing each node data with the item to be searched and return the location of the item in the list if the item found else return null.
8	Traversing	Visiting each node of the list at least once in order to perform some specific operation like searching, sorting, display, etc.

Menu Driven Program: DLL

```
#include<stdio.h>
#include<stdlib.h>
strtct Node
{
        int data;
        struct Node *next, *prev;
};
struct Node *start = NULL, *end = NULL, *temp;
void insertEnd ( int ele)
{
```

```
        temp = (struct Node *) malloc ( sizeof (struct Node ) );
        temp->data = ele;
        temp->next = temp->prev = NULL;
        if ( start == NULL )            // When list is not having any nodes.
        {
                start = end = temp;
        }
        else            // When list is having atleast one node.
        {
                end->next = temp;
                temp->prev = end;
                end = temp;
        }
}
void deleteEnd ( )
{
        if ( start == NULL )            // When list is empty.
        {
                printf ( " The list is empty ");
        }
        else  if (start == end )        // When list is Having only one node.
        {
                start = end = NULL:
        }
        else            // When list is having more than one node.
        {
                temp = end;
                end = end->prev;
                end->next = NULL;
                free ( temp );
        }
}
void insertBeginning ( int ele )
{
        temp = (struct Node *) malloc ( sizeof (struct Node ) );
        temp->data = ele;
        temp->next = NULL;
        if ( start == NULL )            // When list is not having any nodes.
        {
                start = end = temp;
        }
        else            // When list is having atleast one node.
        {
                temp->next = start;
                start->prev = temp;
                start = temp;
        }
}
void deleteBeginning ( )
{
        if ( start == NULL )            // When list is empty.
        {
                printf ( " The list is empty ");
        }
```

```
        else  if (start == end )        // When list is Having only one node.
        {
                start = end = NULL:
        }
        else            // When list is having more than one node.
        {
                temp = start;           // Assiging first node to temp.
                start = start->next;
                start->prev = NULL;
                free ( temp );          // Deleting first node from memory.
        }

}
void insertAtPosition ( int ele, int pos )
{
        struct Node *curr;
        int i;
        if ( pos == 1)
        {
                                        insertBeginning ( ele );
        }
        else
        {
                                        temp = (struct Node *)
                                        malloc ( sizeof (struct
                                        Node ) ); temp->data =
                                        ele;
                                        temp->next = NULL;

                                        curr = start;
                                        for ( i=1; i<pos-1; i++ )       // Moving to
                                        previous of target position
                                        {
                        curr = curr->next;
                }
                if ( curr == NULL )
                {
                                        printf ( " \n Insertion not possible, Position
                }                       out of range " );
                else
                {

                                        te
                                        m
                                        p-
                                        >n
                                        ext
                                }       =
                        }               cu
                }                       rr-
                                        >n
                                        ext
                                        ;
                                        cu
```

```
rr->next->prev = temp;
curr->next = temp->prev = curr;
if (curr == end)
end = temp;
```

```
void deleteAtPosition ( int pos )
{
        int i;
        if ( start == NULL )
        {
                printf ( " List is empty ");
        }
```

```
        else if ( pos == 1)
        {
                                        deleteBeginning ( );
        }
        else
        {
                                        temp = start;
                                        for ( i=1; i<pos-1; i++ )       // Moving to
                                        previous of target position
                                        {
                temp = temp->next;
            }
            if ( temp == NULL )
            {
                printf ( " \n Deletion not possible, Position out of range " );
            }
            else
            {       if ( temp->next == end )
                    {
                                        end = temp;
                                        end->next = NULL:
                    }
                    else
                    {
                                        temp->next =
                                        temp->next->ne
                    }                   xt;
            }                           temp->next->pre
        }                               v = temp;
}

void displayList ( )
{
        if ( start == NULL )
        {
                                        printf ( " List is empty ");
        }
        else
        {
                                        temp = start;
                                        while ( temp != end)            // Diplaying all
                                        nodes except last one.
                                        {
                printf ( " %d <-> ", temp->data );
                temp = temp -> next;
            }
            printf ( " %d ", temp->data );        // To display the last node.
        }
}
void searchKey ( int key )
{
        if ( start == NULL )
        {
            printf ( " List is empty " );
        }
```

```
else
{
```

```
                temp = start;
                while ( temp != NULL )
                {
                        if ( temp->data == key )
                        {
                                printf ( " \n Key node found in the list. ");
                                break;
                        }
                        temp = temp -> next;
                }
                if ( temp == NULL )
                {
                        printf ( " \n Key element not found in the list. ");
                }
        }
}
int main ( )
{
        int choice, ele, pos, key ;
        while ( 1 )
        {
                printf ( " \n Doubly Linked List Operations \n ");

                printf( " 1. Insert End  \n ");
                printf( " 2. Delete End \n ");
                printf( " 3. Insert Beginning \n ");
                printf( " 4. Delete Beginning \n ");
                printf( " 5. Insert At Position \n
                ");
                printf( " 6. Delete At Position \n ");
                printf( " 7. Display List \n ");
                printf( " 8. Search Key \n ");
                printf( " 9. Exit\n ");

                printf ( " Enter your choice: " );
                scanf ( " %d " , &choice );

                switch ( choice )
                {
                        case 1 :
                                printf ( " Enter the element: "
                                ); scanf ( " %d " , &ele);
                                insertEnd ( ele );
                                break;
                        case 2 :
                                deleteEnd ( );
                                break;
                        case 3 :
                                printf ( " Enter the element: " );
                                scanf ( " %d " , &ele);
                                insertBeginning ( ele );
                                break;
                        case 4 :
                                deleteBeginning ( );
                                break;
                        case 5 :
                                printf ( " Enter the element: " );
```

```
scanf ( " %d " , &ele);
```

```
                    printf ( " Enter the position : " );
                    scanf ( " %d ", &pos);
                    insertAtPosition ( ele, pos );
                    break;
            case 6 :
                    printf (" Enter the position: " );
                    scanf ( " %d ", &pos);
                    deleteAtPosition ( pos );
                    break;
            case 7 :
                    displayList ( );
                    break;
            case 8 :
                    printf ( " Enter the key element to find: " );
                    scanf ( " %d ", &key);
                    searchKey ( key );
                    break;
            case 9 :
            default :
                    return 0;
        }
    }
}
```

Applications of linked list in computer science –

1. Implementation of stacks and queues
2. Implementation of graphs: Adjacency list representation of graphs is most popular which is uses linked list to store adjacent vertices.
3. Dynamic memory allocation: We use linked list of free blocks.
4. Maintaining directory of names
5. Performing arithmetic operations on long integers
6. Manipulation of polynomials by storing constants in the node of linked list
7. representing sparse matrices

Applications of linked list in real world-

1. ***Image viewer*** – Previous and next images are linked, hence can be accessed by next and previous button.
2. ***Previous and next page in web browser*** – We can access previous and next url searched in web browser by pressing back and next button since, they are linked as linked list.
3. ***Music Player*** – Songs in music player are linked to previous and next song. you can play songs either from starting or ending of the list.

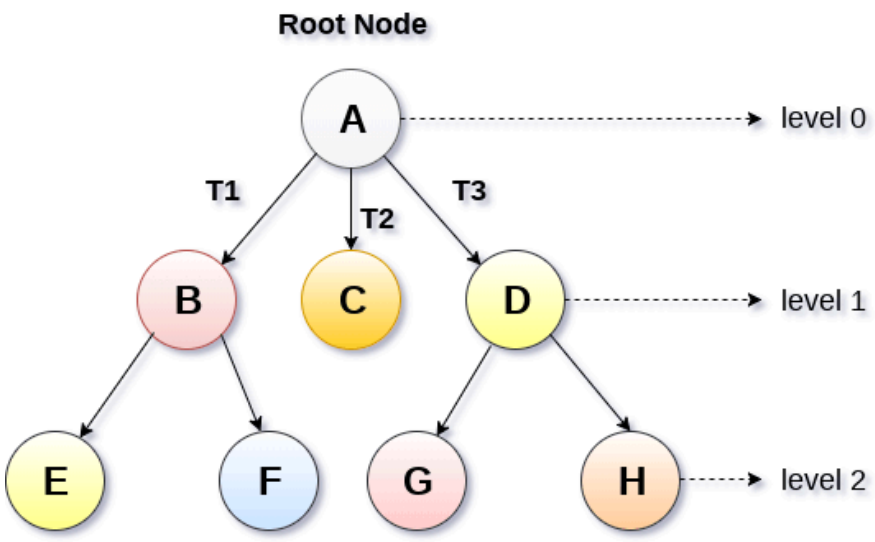
Applications of Circular Linked Lists:

1. Useful for implementation of queue. Unlike this implementation, we don't need to maintain two pointers for front and rear if we use circular linked list. We can maintain a pointer to the last inserted node and front can always be obtained as next of last.

2. Circular lists are useful in applications to repeatedly go around the list. For example, when multiple applications are running on a PC, it is common for the operating system to put the running applications on a list and then to cycle through them, giving each of them a slice of time to execute, and then making them wait while the CPU is given to another application. It is convenient for the operating system to use a circular list so that when it reaches the end of the list it can cycle around to the front of the list.
3. Circular Doubly Linked Lists are used for implementation of advanced data structures like Fibonacci Heap.

Tree

- A Tree is a recursive data structure containing the set of one or more data nodes where one node is designated as the root of the tree while the remaining nodes are called as the children of the root.
- The nodes other than the root node are partitioned into the non empty sets where each one of them is to be called sub-tree.
- Nodes of a tree either maintain a parent-child relationship between them or they are sister nodes.
- In a general tree, A node can have any number of children nodes but it can have only a single parent.
- The following image shows a tree, where the node A is the root node of the tree while the other nodes can be seen as the children of A.



Tree

Basic terminology

- **Root Node** :- The root node is the topmost node in the tree hierarchy. In other words, the root node is the one which doesn't have any parent.
- **Sub Tree** :- If the root node is not null, the tree T1, T2 and T3 is called sub-trees of the root node.
- **Leaf Node** :- The node of tree, which doesn't have any child node, is called leaf node. Leaf node is the bottom most node of the tree. There can be any number of leaf nodes present in a general tree. Leaf nodes can also be called external nodes.
- **Path** :- The sequence of consecutive edges is called path. In the tree shown in the above image, path to the node E is A→ B → E.

- **Ancestor node** :- An ancestor of a node is any predecessor node on a path from root to that node. The root node doesn't have any ancestors. In the tree shown in the above image, the node F have the ancestors, B and A.
- **Degree** :- Degree of a node is equal to number of children, a node have. In the tree shown in the above image, the degree of node B is 2. Degree of a leaf node is always 0 while in a complete binary tree, degree of each node is equal to 2.
- **Level Number** :- Each node of the tree is assigned a level number in such a way that each node is present at one level higher than its parent. Root node of the tree is always present at level 0.

Static representation of tree

```
#define MAXNODE 500
struct treenode {
    int root;
    int father;
    int son;
    int next;
}
```

Dynamic representation of tree

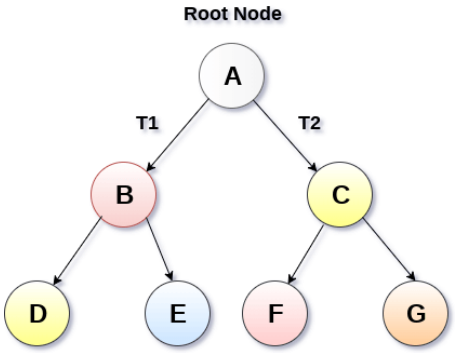
```
struct treenode
{
    int root;
    struct treenode *father;
    struct treenode *son
    struct treenode *next;
}
```

Binary Tree

Binary Tree is a special type of generic tree in which, each node can have at most two children. Binary tree is generally partitioned into three disjoint subsets.

1. Root of the node
2. left sub-tree which is also a binary tree.
3. Right binary sub-tree

A binary Tree is shown in the following image.



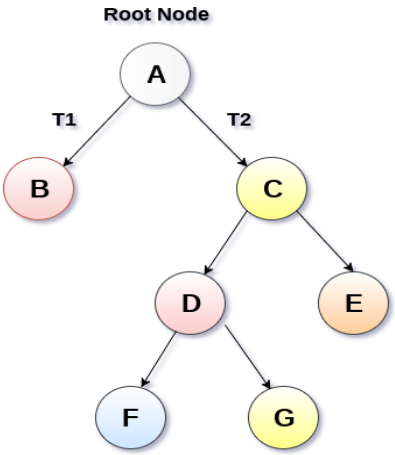
Binary Tree

Types of Binary Tree

1. Strictly Binary Tree

In Strictly Binary Tree, every non-leaf node contain non-empty left and right sub-trees. In other words, the degree of every non-leaf node will always be 2. A strictly binary tree with n leaves, will have (2n - 1) nodes.

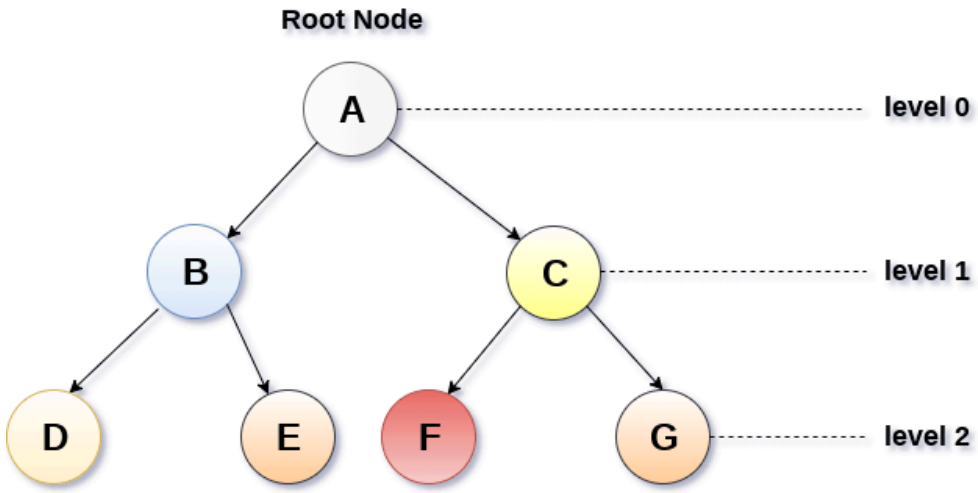
A strictly binary tree is shown in the following figure.



Strictly Binary Tree

2. Complete Binary Tree

A Binary Tree is said to be a complete binary tree if all of the leaves are located at the same level d. A complete binary tree is a binary tree that contains exactly 2^l nodes at each level between level 0 and d. The total number of nodes in a complete binary tree with depth d is 2^{d+1}-1 where leaf nodes are 2^d while non-leaf nodes are 2^d-1.



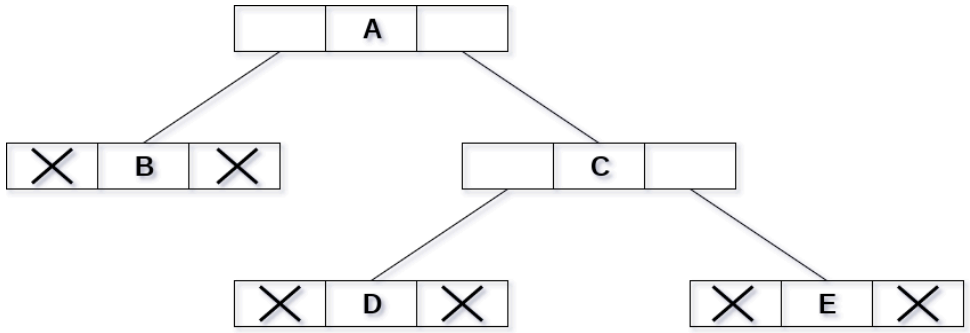
Complete Binary Tree

Binary Tree representation

There are two types of representation of a binary tree:

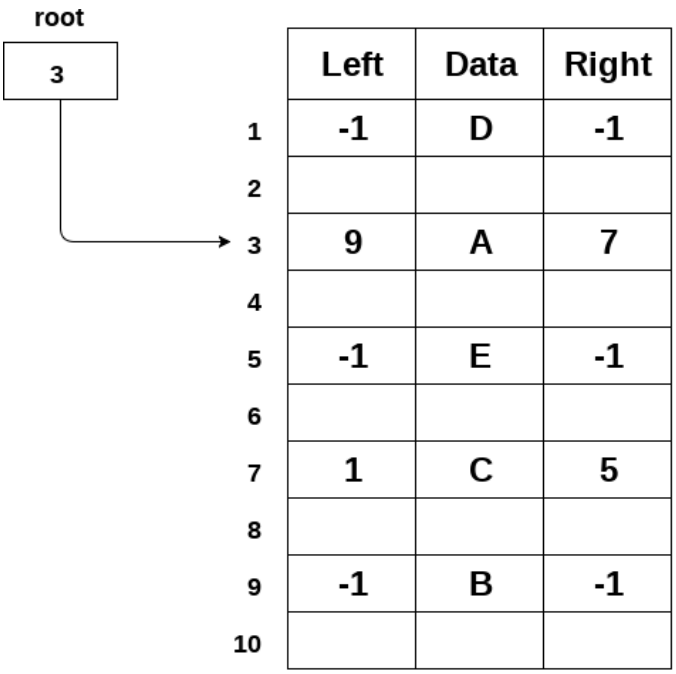
1. Linked Representation

In this representation, the binary tree is stored in the memory, in the form of a linked list where the number of nodes are stored at non-contiguous memory locations and linked together by inheriting parent child relationship like a tree. every node contains three parts: pointer to the left node, data element and pointer to the right node. Each binary tree has a root pointer which points to the root node of the binary tree. In an empty binary tree, the root pointer will point to **NULL**. Consider the binary tree given in the figure below.



In the above figure, a tree is seen as the collection of nodes where each node contains three parts: left pointer, data element and right pointer. Left pointer stores the address of the left child while the right pointer stores the address of the right child. The leaf node contains **null** in its left and right pointers.

The following image shows about how the memory will be allocated for the binary tree by using linked representation. There is a special pointer maintained in the memory which points to the root node of the tree. Every node in the tree contains the address of its left and right child. Leaf node contains null in its left and right pointers.

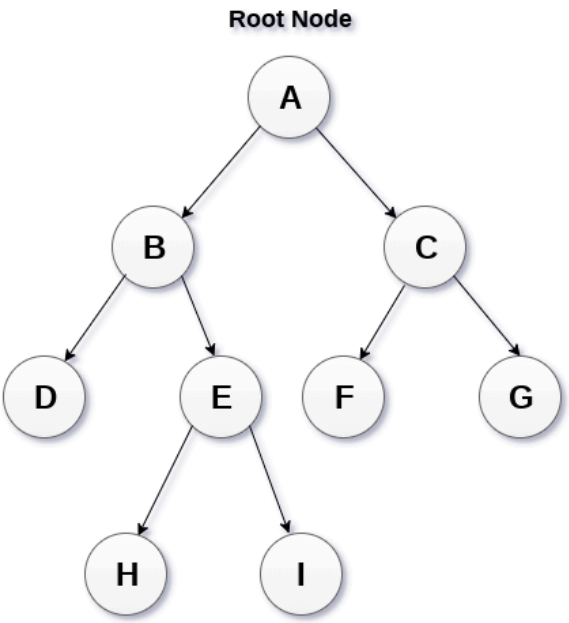


Memory Allocation of Binary Tree using linked Representation

2. Sequential Representation (Array Representation)

This is the simplest memory allocation technique to store the tree elements but it is an inefficient technique since it requires a lot of space to store the tree elements. A binary tree is shown in the following figure along with its memory allocation.

In this representation, an array is used to store the tree elements. Size of the array will be equal to the number of nodes present in the tree. The root node of the tree will be present at the 1st index of the array. If a node is stored at ith index then its left and right children will be stored at 2i and 2i+1 location. If the 1st index of the array i.e. tree[1] is 0, it means that the tree is empty.



A	B	C	D	E	F	G			H	I
1	2	3	4	5	6	7	8	9	10	11

Sequential Representation of Binary Tree

Binary Tree Traversal

Traversal is a process to visit all the nodes of a tree and may print their values too. Because, all nodes are connected via edges (links) we always start from the root (head) node. That is, we cannot randomly access a node in a tree. There are three ways which we use to traverse a tree –

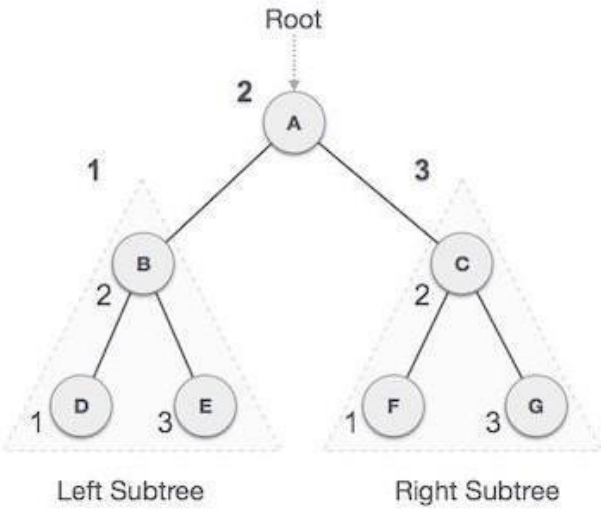
- In-order Traversal
- Pre-order Traversal
- Post-order Traversal

Generally, we traverse a tree to search or locate a given item or key in the tree or to print all the values it contains.

In-order Traversal

In this traversal method, the left subtree is visited first, then the root and later the right sub-tree. We should always remember that every node may represent a subtree itself.

If a binary tree is traversed **in-order**, the output will produce sorted key values in an ascending order.



We start from **A**, and following in-order traversal, we move to its left subtree **B**. **B** is also traversed in-order. The process goes on until all the nodes are visited. The output of inorder traversal of this tree will be –

$$D \rightarrow B \rightarrow E \rightarrow A \rightarrow F \rightarrow C \rightarrow G$$

Algorithm

Until all nodes are traversed –

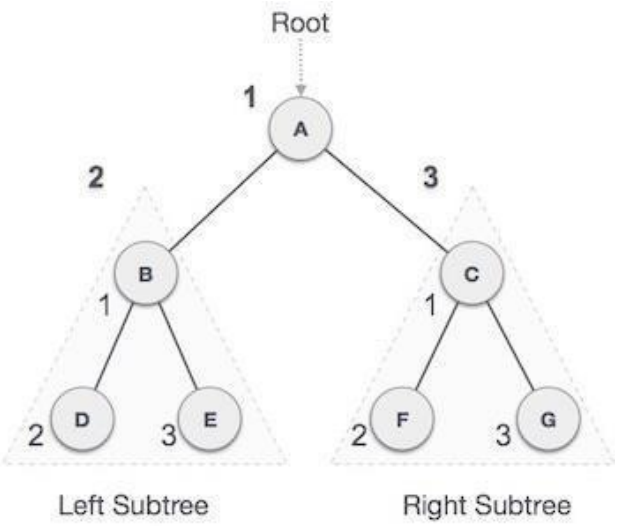
Step 1 – Recursively traverse left subtree.

Step 2 – Visit root node.

Step 3 – Recursively traverse right subtree.

Pre-order Traversal

In this traversal method, the root node is visited first, then the left subtree and finally the right subtree.



We start from **A**, and following pre-order traversal, we first visit **A** itself and then move to its left subtree **B**. **B** is also traversed pre-order. The process goes on until all the nodes are visited. The output of pre-order traversal of this tree will be –

$$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$$

Algorithm

Until all nodes are traversed –

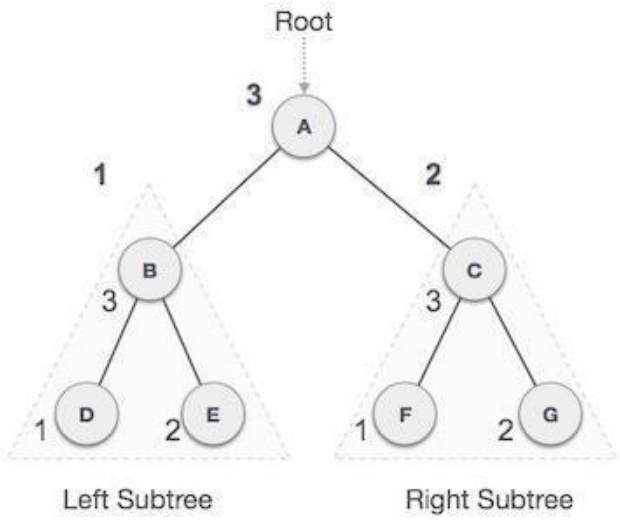
Step 1 – Visit root node.

Step 2 – Recursively traverse left subtree.

Step 3 – Recursively traverse right subtree.

Post-order Traversal

In this traversal method, the root node is visited last, hence the name. First we traverse the left subtree, then the right subtree and finally the root node.



We start from **A**, and following Post-order traversal, we first visit the left subtree **B**. **B** is also traversed post-order. The process goes on until all the nodes are visited. The output of post-order traversal of this tree will be –

$$D \rightarrow E \rightarrow B \rightarrow F \rightarrow G \rightarrow C \rightarrow A$$

Algorithm

Until all nodes are traversed –

Step 1 – Recursively traverse left subtree.

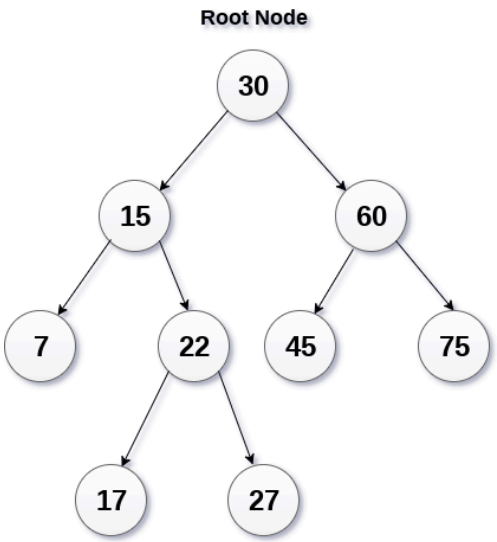
Step 2

– Recursively traverse right subtree.

Step 3 – Visit root node.

Binary Search Tree

1. Binary Search tree can be defined as a class of binary trees, in which the nodes are arranged in a specific order. This is also called ordered binary tree.
2. In a binary search tree, the value of all the nodes in the left sub-tree is less than the value of the root.
3. Similarly, value of all the nodes in the right sub-tree is greater than or equal to the value of the root.
4. This rule will be recursively applied to all the left and right sub-trees of the root.



Binary Search Tree

A Binary search tree is shown in the above figure. As the constraint applied on the BST, we can see that the root node 30 doesn't contain any value greater than or equal to 30 in its left sub-tree and it also doesn't contain any value less than 30 in its right sub-tree.

Advantages of using binary search tree

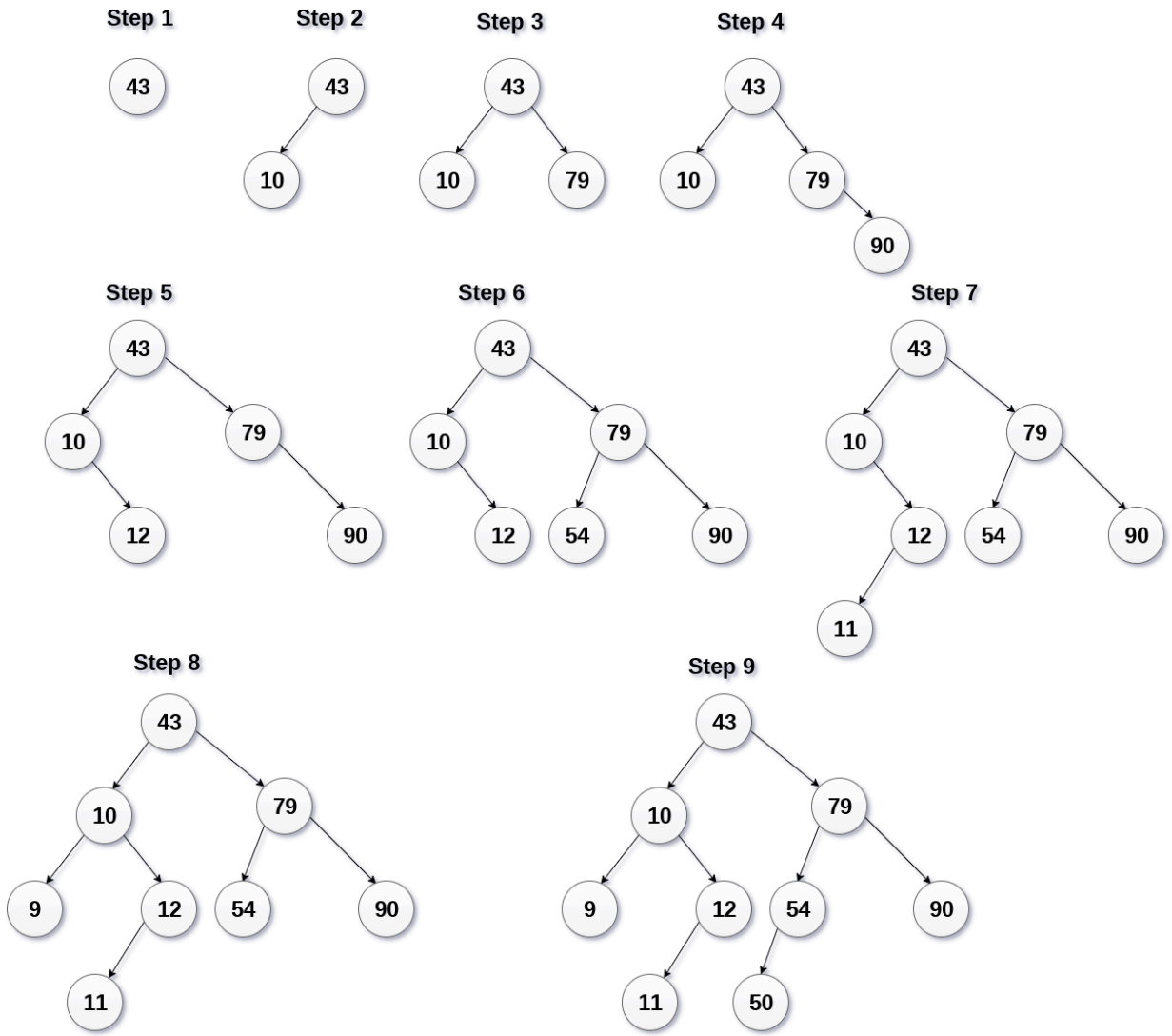
1. Searching become very efficient in a binary search tree since, we get a hint at each step, about which sub-tree contains the desired element.
2. The binary search tree is considered as efficient data structure in compare to arrays and linked lists. In searching process, it removes half sub-tree at every step. Searching for an element in a binary search tree takes o($\log_2 n$) time. In worst case, the time it takes to search an element is 0(n).
3. It also speed up the insertion and deletion operations as compare to that in array and linked list.

Creating the binary search tree using the following data elements.

43, 10, 79, 90, 12, 54, 11, 9, 50

1. Insert 43 into the tree as the root of the tree.
2. Read the next element, if it is lesser than the root node element, insert it as the root of the left sub-tree.
3. Otherwise, insert it as the root of the right of the right sub-tree.

The process of creating BST by using the given elements, is shown in the image below.



Binary search Tree Creation

Operations on Binary Search Tree

There are many operations which can be performed on a binary search tree.

SN	Operation	Description
1	Searching in BST	Finding the location of some specific element in a binary search tree.
2	Insertion in BST	Adding a new element to the binary search tree at the appropriate location so that the property of BST do not violate.
3	Deletion in BST	Deleting some specific node from a binary search tree. However, there can be various cases in deletion depending upon the number of children, the node have.

Program to implement BST operations and Traversals using recursion

```
#include<stdio.h>
#include<stdlib.h>
typedef struct Node
{
	int data;
	struct Node *left, *right;
}bstNode;

bstNode* insert(bstNode *temp, int val)
{
	if(temp == NULL)
	{
		temp = (bstNode*)malloc(sizeof(bstNode));
		temp->data = val;
		temp->left = temp->right = NULL;
	}
	else if(val == temp->data)
		printf("\nNode already existing\n");
	else if(val < temp->data)
		temp->left = insert(temp->left, val);
	else
		temp->right = insert(temp->right, val);
	return temp;
}
void inorder(bstNode *root)
{
	if(root != NULL)
	{
		inorder(root->left);
```

```
                printf("%d ", root->data);
                inorder(root->right);
        }
}
void preorder(bstNode *root)
{
        if(root != NULL)
        {
                printf("%d ", root->data);
                preorder(root->left);
                preorder(root->right);
        }
}
void postorder(bstNode *root)
{
        if(root != NULL)
        {
                postorder(root->left);
                postorder(root->right);
                printf("%d ", root->data);
        }
}
bstNode* findMax(bstNode *temp)
{
        if(temp->right)
                return findMax(temp->right);
        return temp;
}
bstNode* findMin(bstNode *temp)
{
        if(temp->left)
                return findMin(temp->left);
        return temp;
}
bstNode* deleteNode(bstNode *node, int val)
{
        bstNode *temp;
        if(node == NULL)
        {
                printf("\n Node not found");
        }
        else if(val < node->data)
        {
                node->left = deleteNode(node->left, val);
        }
        else if(val > node->data)
        {
```

```
node->right = deleteNode(node->right, val);

        }
        else
        {
                if(node->left && node->right)
                {
                tem
                p =
                find
                Min(
                nod
                e->ri
                ght);
                nod
                e->d
                ata =
                tem
                p->d
                ata;
                node->right = deleteNode(node->right,
                temp->data);

                temp = node;
                if(node->right == NULL)
                        n
                od
                  e
                  =
                no
                de
                ->l
                 ef
                  t;
                 el
                se
                 if(
                no
                de
                ->l
                 ef
                   t
                ==
                  N
                UL
                 L)
                no
                de
        }
        else
        {
                }
                }
```

```
 =                                                    ht
no                                                     ;
de                                                   free(temp);
 ->
rig
                return node;
        }
        int main()
        {
                bstNode *root = NULL, *temp;
                clrscr();
                while(1)
                {
                        int choice, val;
                        printf("\n BST Operations: ");
                        printf("\n 1. Insert \n 2. Inorder \n 3. Preorder \n 4. Postorder");
                        printf("\n 5. Find Min \n 6. Find Max \n 7. Delete \n 8. Exit ");
                        printf("\n Enter your choice: ");
                        scanf("%d", &choice);
                        switch(choice)
                        {
                                        case 1: printf("\n Enter the
                                                value: "); scanf("%d",
                                                &val);
                                                root =
                                                insert(ro
                                        case 2: ot, val);
                                                break;

                                                printf(
                                        case 3: "\n
                                                Inorde
                                                r: ");
                                                inorde
                                                r(root)
                                                ;
                                                break;

                                                printf("\n Preorder: ");
```

```
                    case 4: preorder(root);break;
                    case 5: printf("\n Postorder: ");postorder(root);break;

                    if(root != NULL)
                    {
                        temp = findMin(root);
                        printf("\n Min = %d", temp->data);
                    }
                    break;

                    case 6:

                    if(root != NULL)
                    {
                        temp = findMax(root);
                        printf("\n Max = %d", temp->data);
                    }
                    break;

                    case 7:

                    printf("\n Enter the node to delete: ");
                    scanf("%d", &val);
```

```
                        root = deleteNode(root, val);
                        break;
                default:
                        return 0;
            }
        }
    }
```

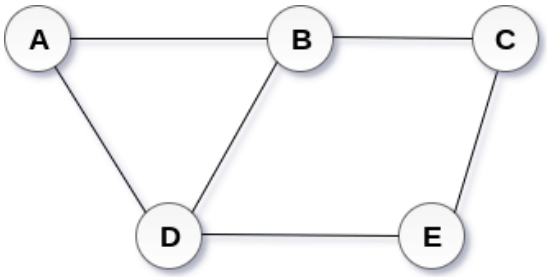
Graph

A graph can be defined as group of vertices and edges that are used to connect these vertices. A graph can be seen as a cyclic tree, where the vertices (Nodes) maintain any complex relationship among them instead of having parent child relationship.

Definition

A graph G can be defined as an ordered set G(V, E) where V(G) represents the set of vertices and E(G) represents the set of edges which are used to connect these vertices.

A Graph G(V, E) with 5 vertices (A, B, C, D, E) and six edges ((A,B), (B,C), (C,E), (E,D), (D,B), (D,A)) is shown in the following figure.



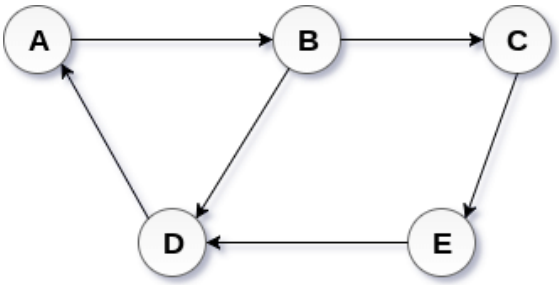
Undirected Graph

Directed and Undirected Graph

A graph can be directed or undirected. However, in an undirected graph, edges are not associated with the directions with them. An undirected graph is shown in the above figure since its edges are not attached with any of the directions. If an edge exists between vertex A and B then the vertices can be traversed from B to A as well as A to B.

In a directed graph, edges form an ordered pair. Edges represent a specific path from some vertex A to another vertex B. Node A is called initial node while node B is called terminal node.

A directed graph is shown in the following figure.



Directed Graph

Graph Terminology

Path

A path can be defined as the sequence of nodes that are followed in order to reach some terminal node V from the initial node U.

Closed Path

A path will be called as closed path if the initial node is same as terminal node. A path will be closed path if $V_0=V_N$.

Simple Path

If all the nodes of the graph are distinct with an exception $V_0=V_N$, then such path P is called as closed simple path.

Cycle

A cycle can be defined as the path which has no repeated edges or vertices except the first and last vertices.

Connected Graph

A connected graph is the one in which some path exists between every two vertices (u, v) in V. There are no isolated nodes in connected graph.

Complete Graph

A complete graph is the one in which every node is connected with all other nodes. A complete graph contain n(n-1)/2 edges where n is the number of nodes in the graph.

Weighted Graph

In a weighted graph, each edge is assigned with some data such as length or weight. The weight of an edge e can be given as w(e) which must be a positive (+) value indicating the cost of traversing the edge.

Digraph

A digraph is a directed graph in which each edge of the graph is associated with some direction and the traversing can be done only in the specified direction.

Loop

An edge that is associated with the similar end points can be called as Loop.

Adjacent Nodes

If two nodes u and v are connected via an edge e, then the nodes u and v are called as neighbours or adjacent nodes.

Degree of the Node

A degree of a node is the number of edges that are connected with that node. A node with degree 0 is called as isolated node.

Graph Representation

By Graph representation, we simply mean the technique which is to be used in order to store some graph into the computer's memory.

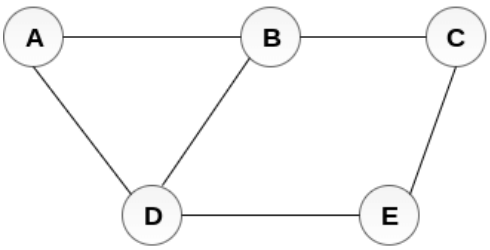
There are two ways to store Graph into the computer's memory. In this part of this tutorial, we discuss each one of them in detail.

1. Sequential Representation (Adjacency Matrix)

In sequential representation, we use adjacency matrix to store the mapping represented by vertices and edges. In adjacency matrix, the rows and columns are represented by the graph vertices. A graph having n vertices, will have a dimension **n x n**.

An entry M_{ij} in the adjacency matrix representation of an undirected graph G will be 1 if there exists an edge between V_i and V_j.

An undirected graph and its adjacency matrix representation is shown in the following figure.



Undirected Graph

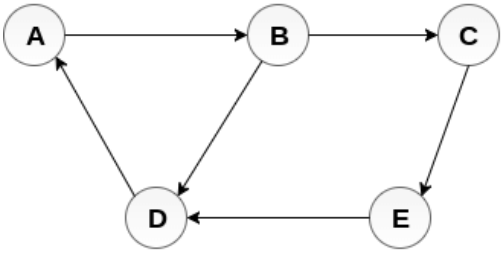
$$\begin{array}{c|ccccc} & A & B & C & D & E \\ \hline A & 0 & 1 & 0 & 1 & 0 \\ B & 1 & 0 & 1 & 1 & 0 \\ C & 0 & 1 & 0 & 0 & 1 \\ D & 1 & 1 & 0 & 0 & 1 \\ E & 0 & 0 & 1 & 1 & 0 \end{array}$$

Adjacency Matrix

in the above figure, we can see the mapping among the vertices (A, B, C, D, E) is represented by using the adjacency matrix which is also shown in the figure.

There exist different adjacency matrices for the directed and undirected graph. In directed graph, an entry A_{ij} will be 1 only when there is an edge directed from V_i to V_j.

A directed graph and its adjacency matrix representation is shown in the following figure.



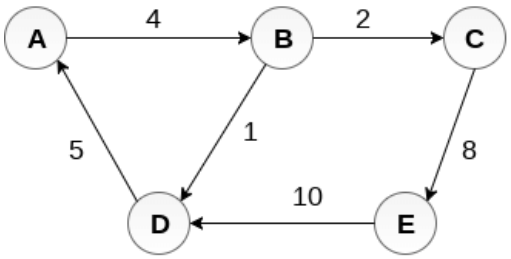
Directed Graph

$$\begin{array}{c|ccccc} & A & B & C & D & E \\ \hline A & 0 & 1 & 0 & 0 & 0 \\ B & 0 & 0 & 1 & 1 & 0 \\ C & 0 & 0 & 0 & 0 & 1 \\ D & 1 & 0 & 0 & 0 & 0 \\ E & 0 & 0 & 0 & 1 & 0 \end{array}$$

Adjacency Matrix

Representation of weighted directed graph is different. Instead of filling the entry by 1, the Non-zero entries of the adjacency matrix are represented by the weight of respective edges.

The weighted directed graph along with the adjacency matrix representation is shown in the following figure.



Weighted Directed Graph

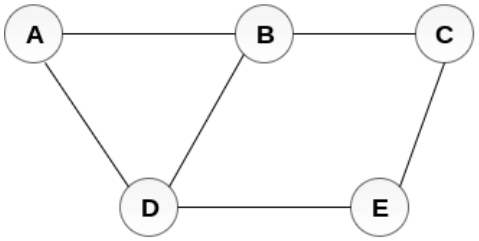
$$\begin{array}{c|ccccc} & A & B & C & D & E \\ \hline A & 0 & 4 & 0 & 0 & 0 \\ B & 0 & 0 & 2 & 1 & 0 \\ C & 0 & 0 & 0 & 0 & 8 \\ D & 5 & 0 & 0 & 0 & 0 \\ E & 0 & 0 & 0 & 10 & 0 \end{array}$$

Adjacency Matrix

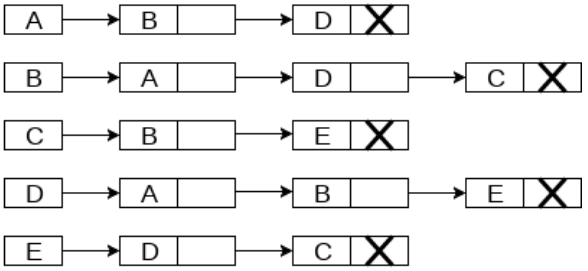
2. Linked Representation (Adjacency List)

In the linked representation, an adjacency list is used to store the Graph into the computer's memory.

Consider the undirected graph shown in the following figure and check the adjacency list representation.



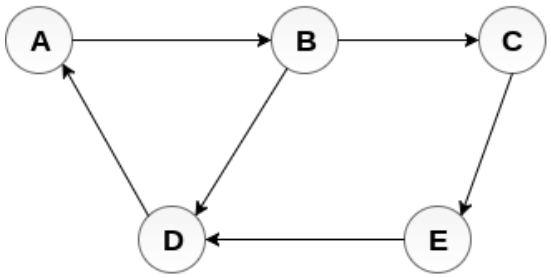
Undirected Graph



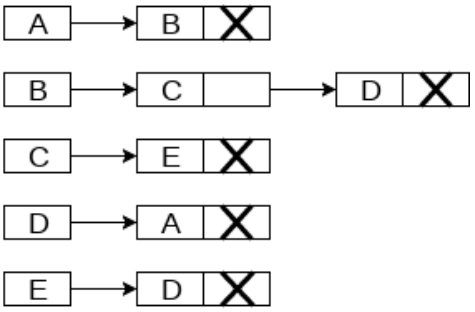
Adjacency List

An adjacency list is maintained for each node present in the graph which stores the node value and a pointer to the next adjacent node to the respective node. If all the adjacent nodes are traversed, then store the NULL in the pointer field of last node of the list. The sum of the lengths of adjacency lists is equal to the twice of the number of edges present in an undirected graph.

Consider the directed graph shown in the following figure and check the adjacency list representation of the graph.



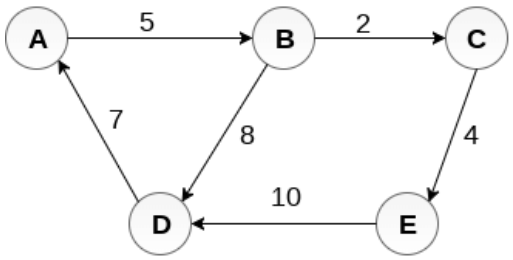
Directed Graph



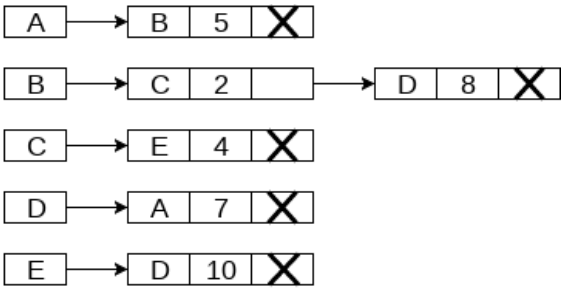
Adjacency List

In a directed graph, the sum of lengths of all the adjacency lists is equal to the number of edges present in the graph.

In the case of weighted directed graph, each node contains an extra field that is called the weight of the node. The adjacency list representation of a directed graph is shown in the following figure.



Weighted Directed Graph



Adjacency List

Graph Traversal Algorithm

In this part of the tutorial we will discuss the techniques by using which, we can traverse all the vertices of the graph.

Traversing the graph means examining all the nodes and vertices of the graph. There are two standard methods by using which, we can traverse the graphs. Let's discuss each one of them in detail.

- Breadth First Search
- Depth First Search

Breadth First Search (BFS) Algorithm

Breadth first search is a graph traversal algorithm that starts traversing the graph from root node and explores all the neighboring nodes. Then, it selects the nearest node and explore all the

unexplored nodes. The algorithm follows the same process for each of the nearest node until it finds the goal.

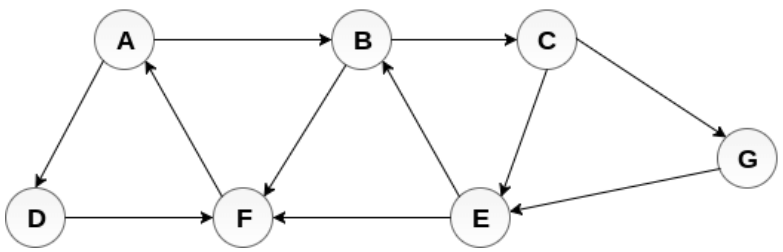
The algorithm of breadth first search is given below. The algorithm starts with examining the node A and all of its neighbors. In the next step, the neighbors of the nearest node of A are explored and process continues in the further steps. The algorithm explores all neighbors of all the nodes and ensures that each node is visited exactly once and no node is visited twice.

Algorithm

- **Step 1:** SET STATUS = 1 (ready state) for each node in G
- **Step 2:** Enqueue the starting node A and set its STATUS = 2 (waiting state)
- **Step 3:** Repeat Steps 4 and 5 until QUEUE is empty
- **Step 4:** Dequeue a node N. Process it and set its STATUS = 3 (processed state).
- **Step 5:** Enqueue all the neighbors of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2 (waiting state) [END OF LOOP]
- **Step 6:** EXIT

Example

Consider the graph G shown in the following image, calculate the minimum path p from node A to node E. Given that each edge has a length of 1.



Adjacency Lists

A : B, D
B : C, F
C : E, G
G : E
E : B, F
F : A
D : F

Solution:

Minimum Path P can be found by applying breadth first search algorithm that will begin at node A and will end at E. the algorithm uses two queues, namely **QUEUE1** and **QUEUE2**. **QUEUE1** holds all the nodes that are to be processed while **QUEUE2** holds all the nodes that are processed and deleted from **QUEUE1**.

Lets start examining the graph from Node A.

1. Add A to QUEUE1 and NULL to QUEUE2.

QUEUE1 = {A}
QUEUE2 = {NULL}

2. Delete the Node A from QUEUE1 and insert all its neighbors. Insert Node A into QUEUE2

QUEUE1 = {B, D}
QUEUE2 = {A}

3. Delete the node B from QUEUE1 and insert all its neighbors. Insert node B into QUEUE2.

QUEUE1 = {D, C, F}
QUEUE2 = {A, B}

4. Delete the node D from QUEUE1 and insert all its neighbors. Since F is the only neighbor of it which has been inserted, we will not insert it again. Insert node D into QUEUE2.

QUEUE1 = {C, F}
QUEUE2 = { A, B, D}

5. Delete the node C from QUEUE1 and insert all its neighbors. Add node C to QUEUE2.

QUEUE1 = {F, E, G}
QUEUE2 = {A, B, D, C}

6. Remove F from QUEUE1 and add all its neighbors. Since all of its neighbors has already been added, we will not add them again. Add node F to QUEUE2.

QUEUE1 = {E, G}
QUEUE2 = {A, B, D, C, F}

7. Remove E from QUEUE1, all of E's neighbors has already been added to QUEUE1 therefore we will not add them again. All the nodes are visited and the target node i.e. E is encountered into QUEUE2.

QUEUE1 = {G}
QUEUE2 = {A, B, D, C, F, E}

Now, backtrack from E to A, using the nodes available in QUEUE2.

The minimum path will be: A → B → C → E.

Program: BFS

```
#include<stdio.h>
int a[20][20], q[20], visited[20], n, i, j, f = 0, r = -1;
void bfs(int v)
{
	for(i = 1; i <= n; i++)
		if(a[v][i] && !visited[i])
			q[++r] = i;
	if(f <= r)
	{
		visited[q[f]] = 1;
		bfs(q[f++]);
	}
}

int main()
{
	int v;
	printf("\n Enter the number of vertices:");
	scanf("%d", &n);

	for(i=1; i <= n; i++)
	{
		q[i] = 0;
		visited[i] = 0;
	}

	printf("\n Enter graph data in matrix form:\n");
	for(i=1; i<=n; i++)
	{
		for(j=1;j<=n;j++)
		{
			scanf("%d", &a[i][j]);
		}
	}

	printf("\n Enter the starting vertex:");
	scanf("%d", &v);
	bfs(v);
	printf("\n The node which are reachable are:\n");
```

```
        for(i=1; i <= n; i++)
        {
                if(visited[i])
                        printf("%d\t", i);
                else
                {                       printf("\n Bfs is not possible. Not
                                        all nodes are reachable"); break;

                }
                    }

        return 0;

    }
```

Depth First Search (DFS) Algorithm

Depth first search (DFS) algorithm starts with the initial node of the graph G, and then goes to deeper and deeper until we find the goal node or the node which has no children. The algorithm, then backtracks from the dead end towards the most recent node that is yet to be completely unexplored.

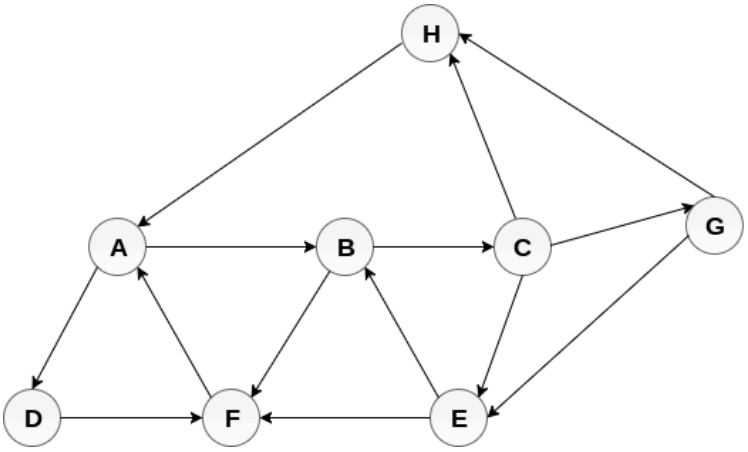
The data structure which is being used in DFS is stack. The process is similar to BFS algorithm. In DFS, the edges that leads to an unvisited node are called discovery edges while the edges that leads to an already visited node are called block edges.

Algorithm

- **Step 1:** SET STATUS = 1 (ready state) for each node in G
- **Step 2:** Push the starting node A on the stack and set its STATUS = 2 (waiting state)
- **Step 3:** Repeat Steps 4 and 5 until STACK is empty
- **Step 4:** Pop the top node N. Process it and set its STATUS = 3 (processed state)
- **Step 5:** Push on the stack all the neighbors of N that are in the ready state (whose STATUS = 1) and set their
  STATUS = 2 (waiting state)
  [END OF LOOP]
- **Step 6:** EXIT

Example:

Consider the graph G along with its adjacency list, given in the figure below. Calculate the order to print all the nodes of the graph starting from node H, by using depth first search (DFS) algorithm.



Adjacency Lists

A : B, D
B : C, F
C : E, G, H
G : E, H
E : B, F
F : A
D : F
H : A

Solution :

1. Push H onto the stack

STACK: H

2. POP the top element of the stack i.e. H, print it and push all the neighbors of H onto the stack that are is ready state.

Print H
STACK: A

3. Pop the top element of the stack i.e. A, print it and push all the neighbors of A onto the stack that are in ready state.

Print A
Stack: B, D

4. Pop the top element of the stack i.e. D, print it and push all the neighbors of D onto the stack that are in ready state.

Print D
Stack: B, F

5. Pop the top element of the stack i.e. F, print it and push all the neighbors of F onto the stack that are in ready state.

Print F
Stack: B

6. Pop the top of the stack i.e. B and push all the neighbors

Print B
Stack: C

7. Pop the top of the stack i.e. C and push all the neighbors.

Print C

Stack: E, G

8. Pop the top of the stack i.e. G and push all its neighbors.

Print G
Stack: E

9. Pop the top of the stack i.e. E and push all its neighbors.

Print E
Stack:

Hence, the stack now becomes empty and all the nodes of the graph have been traversed.

The printing sequence of the graph will be:

H → A → D → F → B → C → G → E

Program: DFS

```
#include<stdio.h>
int a[20][20],reach[20],n;
void dfs(int v)
{
        int i;
        reach[v] = 1;
        for(i=0; i<n; i++)
        {
                if(a[v][i] && !reach[i])
                {
                        printf("\n %d -> %d", v, i);
                        dfs(i);
                }
        }
}
int main()
{
        int i, j, count=0;
        printf("\n Enter number of vertices:");
        scanf("%d", &n);
        for(i=0; i<n; i++)
        {
                reach[i] = 0;
                for(j=0; j<n; j++)
                        a[i][j] = 0;
```

```
        }
        printf("n Enter the adjacency matrix: n");
        for(i=1; i<=n; i++)
                for(j=1; j<=n; j++)
                        scanf("%d", &a[i][j]);
        dfs(1);
        printf("\n");
        for(i=0; i<n;  i++)
        {
                if(reach[i])
                        count++;
        }
        if(count==n-1)
                printf("\n Graph is connected");
        else
                printf("\n Graph is not connected");
        return 0;
}
```